

Deep signature transforms for rough volatility model

1 Preliminary numerical experiments

1.1 Implementation details

We follow Kidger, Bonnier, Perez Arribas, Salvi and Lyons (2019) to design these experiments. Details for each model we used as follows and all models used a sigmoid as a final nonlinearity, so as to map in to $(0, 1)$:

- The Feedforward model was a simple fully connected neural network with 3 hidden layers of 16 neurons each.
- The Neural-Sig model shown in Figure 1 – which is essentially the same model as the Feedforward model, except that the data has the signature applied as feature transformation first – featured hidden layers of sizes 64, 64, 32, 32, 16, 16 respectively.

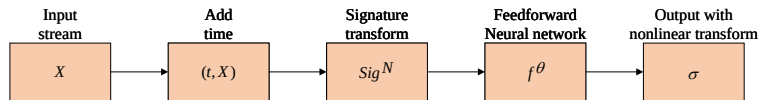


Figure 1: Structure of neural signature model. Trainable parameters: θ .

- The RNN model shown in Figure 2 is two recurrent neural networks, the first comprised of dense layers of sizes 64, 64, 32 and output size 6, and the second comprised of dense layers of size 32, 32, 32, and output size 5. The first network sweeps across the input data relatively slowly, with a stride of 2, whilst the second network sweeps across the result of the first network more quickly, with a stride of 4. In this way it may capture information from the input data at multiple timescales; part of the challenge of fractional Brownian motion is the existence of long-range dependencies.

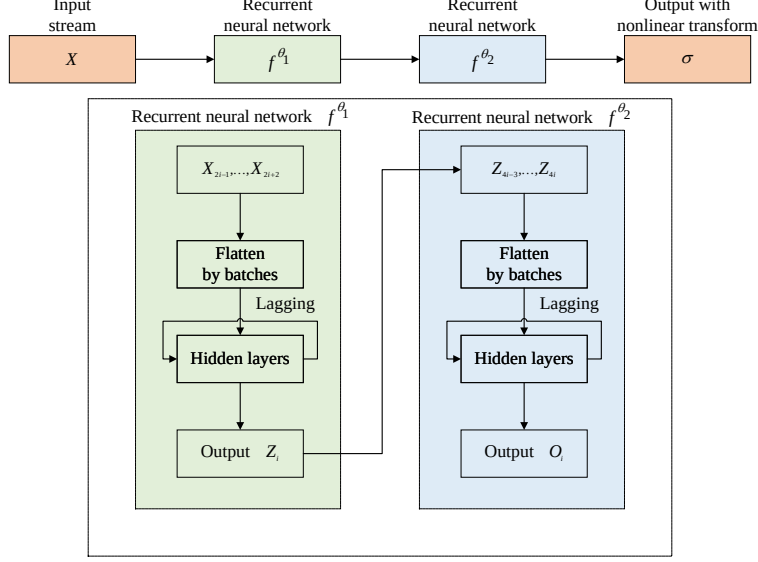


Figure 2: Structure of RNN model. Trainable parameters: θ_1 and θ_2 .

- The LSTM and GRU models both featured two recurrent layers each of size 32, and swept across the raw data with a stride of 1.
- DeepSigNet model shown in Figure 3 featured a single Neural-Lift-Signature block, where the neural component was given by a single convolutional layer with 3 channels and kernel size 3, the lift was the trivial lift, and the signature was truncated as $N = 3$. The neural component also preserved the original time-augmented stream of data, so that in some sense the neural component has 3 extra channels corresponding to time and value. On top of this a feedforward neural network with 5 hidden layers of size 32 was placed. Thus this model is very similar to the Neural-Sig model, except that a small learnable transformation was allowed before the signature. The difference in their performance highlights the value of learning a transformation before using the signature. (Without which the Neural-Sig model is merely outperformed by some non-signature based models.)

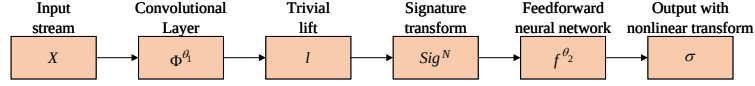


Figure 3: Structure of DeepSigNet model. Trainable parameters: θ_1 and θ_2 .

- DeeperSigNet featured three Neural-Lift-Signature blocks. The neural

component of the first block was a small feedforward network with 2 hidden layers of size 16 and an output layer of size 3, swept across the length of the stream; its kernel size (how many time-value pairs of the stream it saw at once) was 4. The original time-augmented stream of data was also preserved by the neural component. The neural components of the other two blocks were recurrent neural networks, featuring 2 hidden layers of 16 neurons each. The lifts were in every case expanding windows as $l(X) = (X_2, \dots, X_n) = ((x_1, x_2), (x_1, x_2, x_3), \dots, (x_1, x_2, \dots, x_n))$ where $X = (x_1, \dots, x_n) \in \mathcal{S}(\mathbb{R}^d)$. On top of this another recurrent neural network was placed, and the value of its final hidden state used as the output. This final network used 2 hidden layers of 16 neurons each.

The training set featured 600 samples whilst the test set featured 100 samples, each of an instance of fractional Brownian motion sampled at 300 time steps of $[0, 1]$ and generated by Davies-Harte method. These were split up into batches of 128 samples, so the last batch is slightly smaller than the others, and every model trained for 100 epochs.

1.2 Supervised learning with fractional Brownian motion

1.2.1 Reproducing the experiment of the original paper

In this example, we reproduce the second experiment of Kidger, Bonnier, Perez Arribas, Salvi and Lyons (2019) to train a variety of models to estimate Hurst parameter H from paths of fractional Brownian motion B^H . That is, to learn the map

$$((t_0, B_{t_0}^H), \dots, (t_n, B_{t_n}^H)) \mapsto H.$$

Here we set $H \in [0.2, 0.8]$ for data sets and every model trained for 100 epochs. The results are shown in Table 1 and Figure 4. It seems that we did not successfully reproduce the performance gap between the DeepSigNet model and DeeperSigNet model. In addition, the variance of the MSE we obtained is also significantly smaller than the results in the original paper.

Table 1: Final test mean squared error (MSE) for different models, averaged over 3 training runs, ordered from largest to smallest.

	Test MSE		#Params	CPU time
	Mean	Variance		
LSTM	3.76×10^{-2}	1.40×10^{-5}	12961	372.00s
Feedforward	3.21×10^{-2}	1.18×10^{-5}	10209	2.05s
Neural-Sig	1.37×10^{-2}	2.49×10^{-7}	10097	2.48s
RNN	6.40×10^{-3}	1.11×10^{-5}	10091	119.86s
GRU	1.32×10^{-3}	1.73×10^{-7}	9729	291.72s
DeepSigNet	3.25×10^{-4}	5.25×10^{-8}	9261	48.47s
DeeperSigNet	3.21×10^{-4}	3.52×10^{-8}	9686	1500.92s

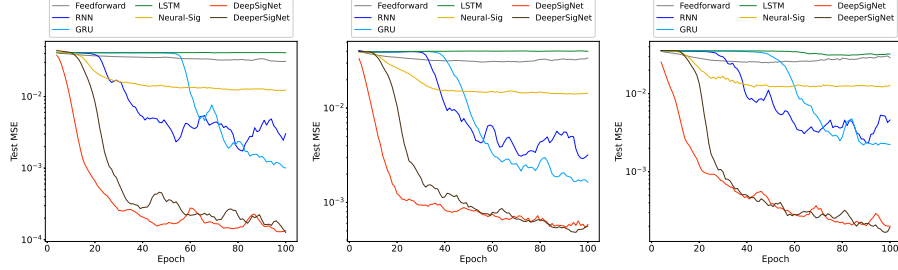


Figure 4: Performance at estimating the Hurst parameter for various models, with and without signatures, for three training runs.

1.2.2 Adjusting Hurst Parameter Range

In this example, we slightly modify the previous experiment by adjusting the range of the Hurst parameter value, while keeping all other settings the same. The results are shown in Table 2 and Figure 5. The results shows that the DeepSigNet model and DeeperSigNet model performs better for $H \in [0.3, 0.5]$, suggesting that paths are smoother compared to those with $H \in (0, 0.2]$.

Table 2: Final test mean squared error (MSE) for different models, averaged over 3 training runs.

	Hurst parameter	Test MSE		CPU time	#Params
		Mean	Variance		
Feedforward	(0, 0.2]	5.87×10^{-3}	5.77×10^{-7}	1.92s	10209
	[0.3, 0.5]	4.85×10^{-3}	1.45×10^{-7}	1.91s	
	(0.5, 0.7]	3.78×10^{-3}	2.17×10^{-7}	1.89s	
RNN	(0, 0.2]	6.03×10^{-3}	5.81×10^{-5}	118.49s	10091
	[0.3, 0.5]	1.02×10^{-3}	5.73×10^{-8}	143.84s	
	(0.5, 0.7]	3.87×10^{-3}	5.42×10^{-8}	143.12s	
DeepSigNet	(0, 0.2]	1.17×10^{-4}	3.53×10^{-10}	47.78s	9261
	[0.3, 0.5]	8.71×10^{-5}	2.24×10^{-10}	47.91s	
	(0.5, 0.7]	2.20×10^{-4}	7.63×10^{-10}	56.74s	
DeeperSigNet	(0, 0.2]	1.01×10^{-4}	9.15×10^{-11}	1537.36s	9686
	[0.3, 0.5]	9.69×10^{-5}	4.12×10^{-11}	1499.44s	
	(0.5, 0.7]	2.74×10^{-4}	3.81×10^{-9}	1508.72s	
LSTM	(0, 0.2]	3.82×10^{-3}	2.37×10^{-7}	205.67s	12961
	[0.3, 0.5]	4.53×10^{-3}	1.54×10^{-7}	454.39s	
	(0.5, 0.7]	3.79×10^{-3}	3.54×10^{-8}	410.67s	
GRU	(0, 0.2]	2.21×10^{-4}	1.79×10^{-10}	251.83s	9729
	[0.3, 0.5]	3.87×10^{-3}	9.08×10^{-7}	326.70s	
	(0.5, 0.7]	3.83×10^{-3}	3.55×10^{-8}	291.88s	
Neural-Sig	(0, 0.2]	3.13×10^{-3}	7.45×10^{-8}	2.27s	10097
	[0.3, 0.5]	4.58×10^{-3}	4.36×10^{-7}	2.25s	
	(0.5, 0.7]	3.23×10^{-3}	2.78×10^{-8}	2.24s	

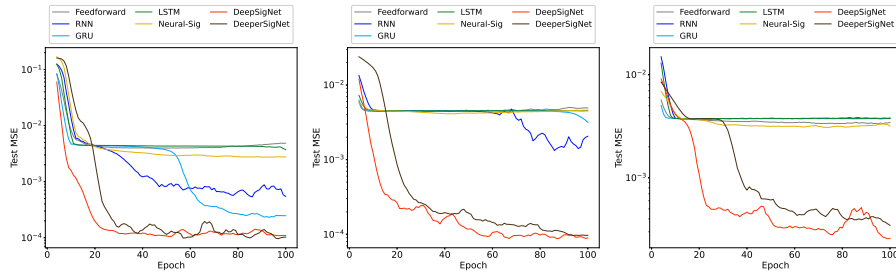


Figure 5: Performance at estimating the Hurst parameter for various models, with and without signatures, for a particular (typical) training run. Here, $H \in (0, 0.2]$ (left); $H \in [0.3, 0.5]$ (middle); $H \in (0.5, 0.7]$ (right).

1.2.3 Adjusting signature truncation order

In this example, we try to adjust the signature truncation order for DeepSigNet model, DeeperSigNet model and Neural-Sig model. The results are shown in Table 3 and Figure 6. In the case of the Neural-Sig model, the test MSE decreases with the signature truncation order increases. However, for the DeepSigNet model and the DeeperSigNet model, selecting a signature truncation order of $N = 3$ or $N = 4$ is sufficient.

Table 3: Final test mean squared error (MSE) for different models, averaged over 3 training runs. In this table, 'null' indicates that the corresponding experiment was not conducted.

	Test MSE		CPU time	#Params	Truncation order
	Mean	Variance			
DeepSigNet	2.90×10^{-3}	1.14×10^{-8}	10.11s	4461	1
DeeperSigNet	1.14×10^{-3}	2.79×10^{-8}	324.07s	2486	
Neural-Sig	4.09×10^{-3}	6.84×10^{-8}	2.23s	8305	
DeepSigNet	1.15×10^{-4}	2.26×10^{-10}	17.72s	5261	2
DeeperSigNet	1.23×10^{-4}	2.18×10^{-9}	546.85s	3686	
Neural-Sig	4.22×10^{-3}	1.31×10^{-8}	2.15s	8561	
DeepSigNet	1.32×10^{-4}	1.39×10^{-10}	48.20s	9261	3
DeeperSigNet	1.49×10^{-4}	1.06×10^{-9}	1495.74s	9686	
Neural-Sig	3.04×10^{-3}	4.65×10^{-7}	2.16s	9073	
DeepSigNet	8.51×10^{-5}	8.74×10^{-11}	237.15s	29261	4
DeeperSigNet	1.11×10^{-4}	1.03×10^{-9}	7361.82s	39686	
Neural-Sig	2.76×10^{-3}	1.57×10^{-7}	2.35s	10097	
DeepSigNet	1.69×10^{-4}	2.94×10^{-9}	1388.30	129261	5
DeeperSigNet	null	null	null	null	
Neural-Sig	1.83×10^{-3}	3.39×10^{-8}	2.28	12145	
DeepSigNet	2.76×10^{-4}	8.75×10^{-9}	8629.09s	629261	6
DeeperSigNet	null	null	null	null	
Neural-Sig	1.55×10^{-3}	6.07×10^{-9}	4.28s	16241	

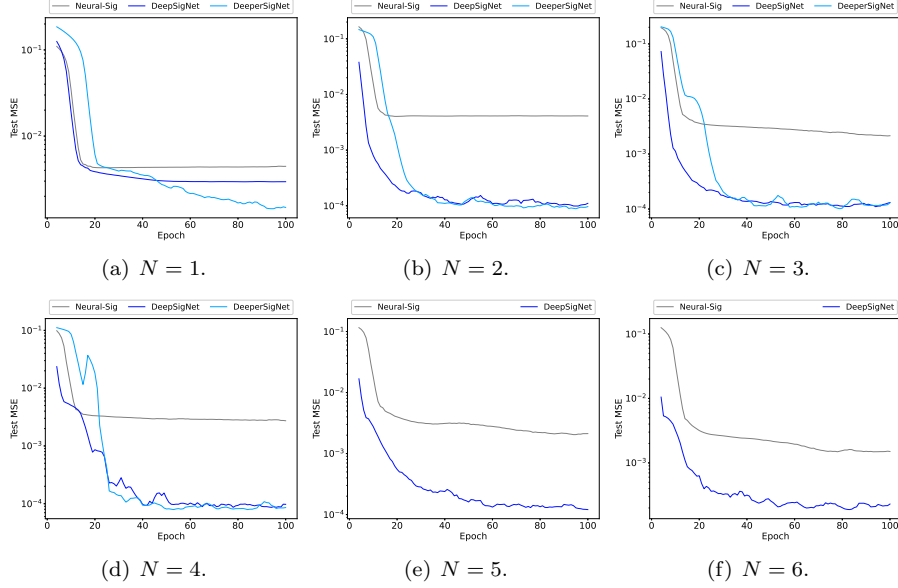


Figure 6: Performance at estimating the Hurst parameter with different signature truncation order N for various models, with and without signatures, for a particular (typical) training run.

1.3 Supervised learning with rough Heston volatility model

In this example, we use the rough Heston volatility model instead of the fractional Brownian motion to generate data sets.

$$\begin{aligned}
V_t &= V_0 + \frac{1}{\Gamma(\frac{1}{2} + H)} \int_0^t (t-s)^{H-\frac{1}{2}} \kappa_1 (\theta - V_s) ds \\
&\quad + \frac{1}{\Gamma(\frac{1}{2} + H)} \int_0^t (t-s)^{H-\frac{1}{2}} \kappa_2 \epsilon \sqrt{V_s} dB_s, \quad t \in [0, T],
\end{aligned}$$

where $\kappa_1, \kappa_2, \theta, \epsilon, V_0, T$ are all constants and B_t is standard Brownian motion. In order to simulate this volatility process, we apply the fast algorithm proposed by Ma and Wu (2022) with uniform mesh $0 = t_0 < \dots < t_n = T$, where N is a positive integer. For comparison, we also introduce another simulation scheme proposed by (Markovian approximation) Bayer and Breneis (2023). Then we train a variety of models to learn the map from these realization data streams of volatility V to Hurst parameter, i.e.

$$((t_0, V_{t_0}), \dots, (t_n, V_{t_n})) \mapsto H.$$

1.3.1 Gaussian Volterra processes

We first set $\kappa_1 = 0$ and remove $\sqrt{V}$ in the Ito integral i.e. the rough Heston volatility process become the Gaussian Volterra process. The other parameters

are set as $\kappa_2 = 0.3$, $\epsilon = 0.3$ and $V_0 = 0$. All other settings are the same as illustration in 1.1. The results are shown as Table 4 and Figure 7.

Table 4: Final test mean squared error (MSE) for different models, averaged over 3 training runs.

	Fast Algorithm			Markovian Approximation			
	Test MSE			Test MSE			#Params
	Mean	Variance	CPU time	Mean	Variance	CPU time	
Feedforward	3.82×10^{-3}	1.26×10^{-7}	2.97s	3.77×10^{-3}	8.09×10^{-9}	2.03s	10209
RNN	3.77×10^{-3}	1.32×10^{-7}	137.91s	3.77×10^{-3}	7.53×10^{-8}	129.49s	10091
DeepSigNet	3.80×10^{-4}	3.51×10^{-9}	56.42s	5.45×10^{-4}	1.64×10^{-8}	48.38s	9261
DeeperSigNet	4.17×10^{-4}	2.41×10^{-8}	1504.62s	6.76×10^{-4}	2.51×10^{-8}	1502.50s	9686
LSTM	3.76×10^{-3}	1.25×10^{-7}	399.85s	3.78×10^{-3}	7.06×10^{-8}	182.08s	12961
GRU	3.76×10^{-3}	1.03×10^{-7}	275.33s	3.80×10^{-3}	6.23×10^{-8}	261.49s	9729
Neural-Sig	3.74×10^{-3}	1.13×10^{-7}	2.30s	3.57×10^{-3}	6.47×10^{-9}	2.24s	10097

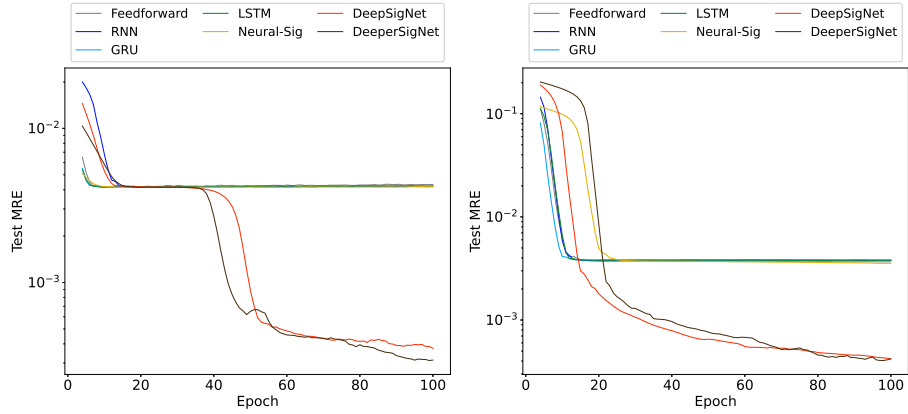


Figure 7: Performance at estimating the Hurst parameter for various models, with and without signatures, for a particular (typical) training run. Here, Fast Algorithm (left); Markovian Approximation (right).

The results appear to differ from those in 1.2.1. To explain this, we propose the paths of fractional Brownian motion and Gaussian Volterra process generated using different methods, as illustrated in Figure 8. In contrast to fractional Brownian motion, when generating the rough Heston volatility process, non-negativity of values is required. If we allow the values of the Gaussian Volterra process to be negative and keep the same amplitude, the results would be consistent with those in 1.2.1.

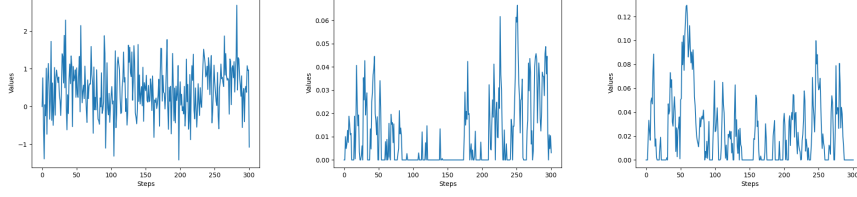


Figure 8: Paths of fractional Brownian motion and Gaussian Volterra process generated by Davies-Harte method (left), Fast Algorithm (middle) and Markovian Approximation (right).

Next, we modify the Gaussian Volterra process by adding the $\sqrt{V}$ in the Ito integral and rerun this experiment. The results are shown as Table 5 and Figure 9.

Table 5: Final test mean squared error (MSE) for different models, averaged over 3 training runs.

	Fast Algorithm			Markovian Approximation			
	Test MSE		CPU time	Test MSE		CPU time	#Params
	Mean	Variance		Mean	Variance		
Feedforward	4.05×10^{-3}	4.92×10^{-8}	4.61s	4.11×10^{-3}	3.49×10^{-9}	1.91s	10209
RNN	4.03×10^{-3}	5.06×10^{-8}	285.59s	4.14×10^{-3}	2.16×10^{-9}	136.08s	10091
DeepSigNet	2.67×10^{-4}	2.27×10^{-9}	62.68s	1.85×10^{-4}	3.73×10^{-10}	48.49s	9261
DeeperSigNet	2.40×10^{-4}	8.88×10^{-10}	2084.20s	2.18×10^{-4}	9.57×10^{-9}	1499.33s	9686
LSTM	3.99×10^{-3}	9.56×10^{-8}	383.10s	4.08×10^{-3}	1.67×10^{-9}	193.08s	12961
GRU	4.01×10^{-3}	7.89×10^{-8}	305.61s	4.08×10^{-3}	1.67×10^{-9}	245.32s	9729
Neural-Sig	4.00×10^{-3}	7.93×10^{-8}	2.36s	4.06×10^{-3}	3.89×10^{-9}	2.36s	10097

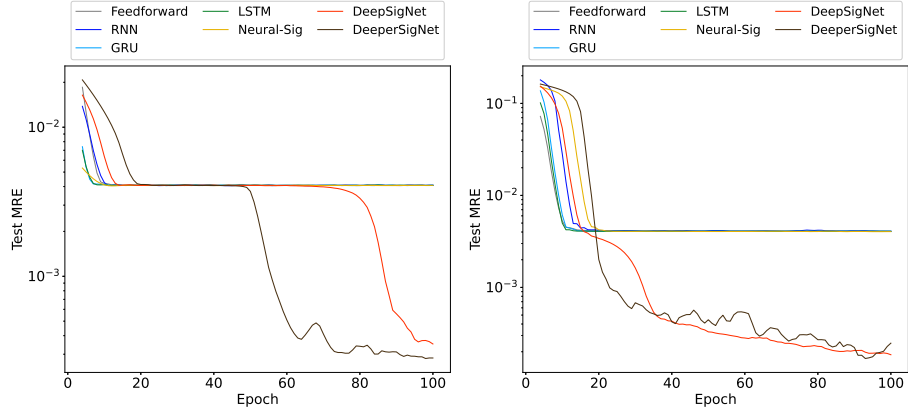


Figure 9: Performance at estimating the Hurst parameter for various models, with and without signatures, for a particular (typical) training run. Here, Fast Algorithm (left); Markovian Approximation (right).

1.3.2 Single parameter calibration for rough Heston volatility processes

Now, we consider the rough Heston volatility process. The parameters are set as $\kappa_1 = \kappa_2 = 0.3$, $\theta = 0.3$, $\epsilon = 0.3$ and $V_0 = 0$. The results are shown as Table 6 and Figure 10.

Table 6: Final test mean squared error (MSE) for different models, averaged over 3 training runs.

	Fast Algorithm			Markovian Approximation			
	Test MSE			Test MSE			#Params
	Mean	Variance	CPU time	Mean	Variance	CPU time	
Feedforward	3.96×10^{-3}	4.88×10^{-8}	3.66s	3.74×10^{-3}	4.29×10^{-9}	1.95s	10209
RNN	3.96×10^{-3}	2.90×10^{-8}	265.48s	4.25×10^{-3}	1.77×10^{-7}	140.18s	10091
DeepSigNet	2.86×10^{-4}	1.22×10^{-8}	94.18s	1.46×10^{-4}	4.54×10^{-10}	84.93s	9261
DeeperSigNet	2.50×10^{-4}	1.86×10^{-9}	1767.80s	1.83×10^{-4}	7.02×10^{-9}	1533.53s	9686
LSTM	3.96×10^{-3}	3.88×10^{-8}	435.44s	4.23×10^{-3}	1.68×10^{-7}	175.07s	12961
GRU	3.97×10^{-3}	3.86×10^{-8}	320.01s	4.21×10^{-3}	1.60×10^{-7}	243.48s	9729
Neural-Sig	3.96×10^{-3}	3.88×10^{-8}	2.38s	3.97×10^{-3}	1.08×10^{-7}	2.39s	10097

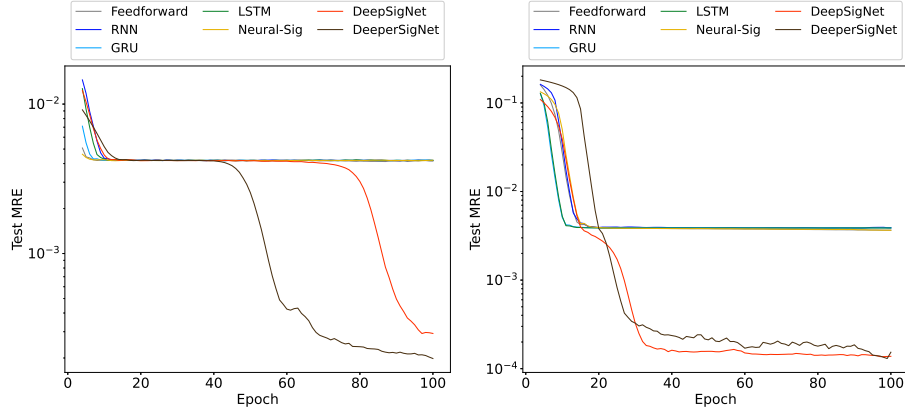


Figure 10: Performance at estimating the Hurst parameter for various models, with and without signatures, for a particular (typical) training run. Here, Fast Algorithm (left); Markovian Approximation (right).

Furthermore, we try to calibrate other parameters in rough Heston volatility processes such as κ_2 . We use the fast algorithm to generate data set with $\kappa_2 \in [0.1, 0.5]$, $\kappa_1 = 0.3$, $\theta = 0.3$, $\epsilon = 0.3$, $H = 0.01$ and $V_0 = 0$. The results in Table 7 and Figure 11 clearly show that the DeepSigNet and DeeperSigNet outperform the other models, similar to the results in calibrating the Hurst parameter.

Table 7: Final test mean squared error (MSE) for different models, averaged over 3 training runs, ordered from largest to smallest.

	Test MSE		#Params	CPU time
	Mean	Variance		
RNN	6.51×10^{-3}	7.00×10^{-7}	10091	155.18s
LSTM	5.27×10^{-3}	1.44×10^{-7}	12961	590.72s
GRU	4.61×10^{-3}	1.06×10^{-7}	9729	366.06s
Neural-Sig	3.20×10^{-3}	1.60×10^{-7}	10097	2.24s
Feedforward	3.03×10^{-3}	1.77×10^{-7}	10209	1.88s
DeeperSigNet	4.95×10^{-4}	7.95×10^{-8}	9686	2448.63s
DeepSigNet	3.10×10^{-4}	1.70×10^{-9}	9261	139.76s

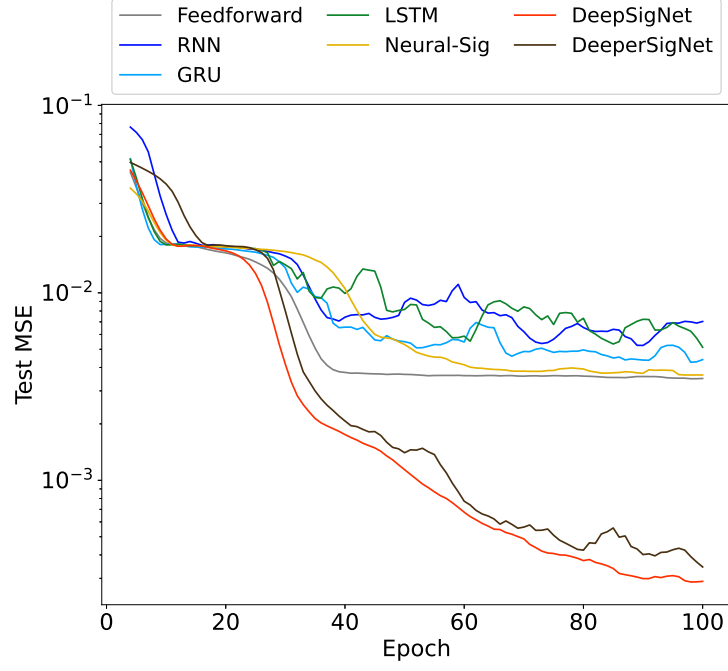


Figure 11: Performance at estimating the κ_1 for various models, with and without signatures, for a particular (typical) training run.

1.3.3 Multi-parameter calibration for rough Heston volatility processes

In this example, we try to calibrate κ_2 and the Hurst parameter simultaneously. We use the fast algorithm to generate data set with $\kappa_2 \in [0.1, 0.5]$, $H \in (0, 0.2]$, $\kappa_1 = 0.3$, $\theta = 0.3$, $\epsilon = 0.3$ and $V_0 = 0$. The results in Table 8 and Figure 12 show that the DeepSigNet and DeeperSigNet outperform the other models but the difference is not significant. We also attempted a higher signature truncation order, but there was no significant improvement.

Table 8: Final test mean squared error (MSE) for different models, averaged over 3 training runs, ordered from largest to smallest.

	Test MSE		#Params	CPU time
	Mean	Variance		
LSTM	1.13×10^{-2}	1.40×10^{-6}	12961	405.41s
GRU	1.01×10^{-2}	7.18×10^{-6}	9729	471.15s
RNN	8.90×10^{-3}	6.97×10^{-6}	10091	124.50s
Neural-Sig	8.58×10^{-3}	8.66×10^{-7}	10097	2.24s
Feedforward	8.01×10^{-3}	3.40×10^{-7}	10209	1.87s
DeepSigNet	6.34×10^{-3}	1.22×10^{-7}	9261	47.86s
DeeperSigNet	6.18×10^{-3}	6.21×10^{-8}	9686	2121.87s

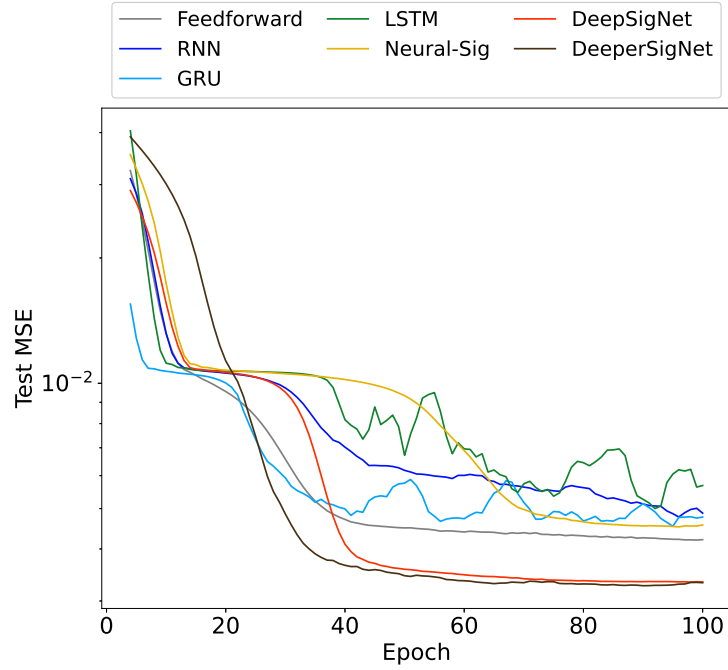


Figure 12: Performance at estimating the κ_1 and H for various models, with and without signatures, for a particular (typical) training run.

1.4 Supervised learning with rough Bergomi volatility model

In this example, we use the rough Bergomi (rBergomi) volatility process instead of rough Heston volatility process to generate data sets. The rBergomi volatility process is defined as

$$V_t = V_0 \exp \left(\int_0^t \eta \sqrt{2H} (t-s)^{H-\frac{1}{2}} dW_s - \frac{\eta^2}{2} t^{2H} \right), V_0 > 0,$$

where W is a standard Brownian motion and η is a strictly positive constant. Here we follow Stone (2020) to use simulated sample paths of the normalized log volatility process $(\log(V_t/V_0))_{t \geq 0}$ of the rBergomi model as the input data.

1.4.1 Single parameter calibration for rough Bergomi volatility processes

We first set $H \in (0, 0.2]$ and $\eta = 1$ to generate data sets. The results in Table 9 and Figure 13 show that the DeepSigNet and DeeperSigNet outperform the other models. This result is similar to the result in Stone (2020), where a convolutional neural network was used with the root mean square error (RMSE) as the measure.

Table 9: Final test mean squared error (MSE) for different models, averaged over 3 training runs, ordered from largest to smallest.

	Test MSE		#Params	CPU time
	Mean	Variance		
Feedforward	2.67×10^{-2}	1.06×10^{-6}	10209	3.47s
LSTM	2.25×10^{-2}	1.37×10^{-4}	12961	246.62s
Neural-Sig	7.10×10^{-3}	8.89×10^{-7}	10097	2.40s
RNN	3.67×10^{-3}	7.87×10^{-7}	10091	108.44s
GRU	2.09×10^{-3}	3.41×10^{-7}	9729	259.43s
DeepSigNet	3.32×10^{-4}	5.19×10^{-8}	9261	48.18s
DeeperSigNet	3.16×10^{-4}	7.13×10^{-9}	9686	1669.37s

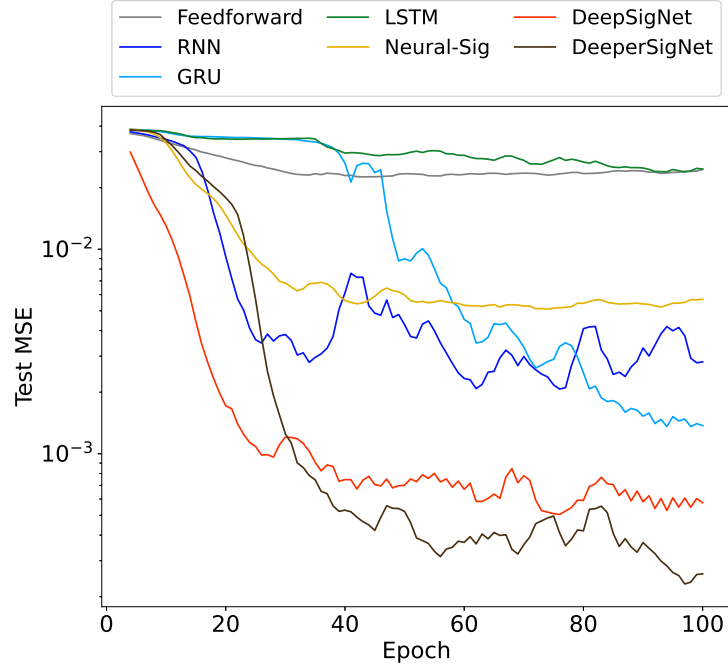


Figure 13: Performance at estimating the Hurst parameter for various models, with and without signatures, for a particular (typical) training run.

References

- Bayer, C., and Breneis, S. (2023). Efficient option pricing in the rough Heston model using weak simulation schemes. arXiv preprint arXiv:2310.04146.
- Kidger, P., Bonnier, P., Perez Arribas, I., Salvi, C., and Lyons, T. (2019). Deep signature transforms. Advances in Neural Information Processing Systems, 32.
- Ma, J. and Wu, H. (2022). A fast algorithm for simulation of rough volatility models. Quantitative Finance, 22, 447–462.
- Stone, H. (2020). Calibrating rough volatility models: a convolutional neural network approach. Quantitative Finance, 20(3), 379–392.